Numerics & boundary conditions

palm group

Institute of Meteorology and Climatology, Leibniz Universität Hannover

Numerics & boundary conditions

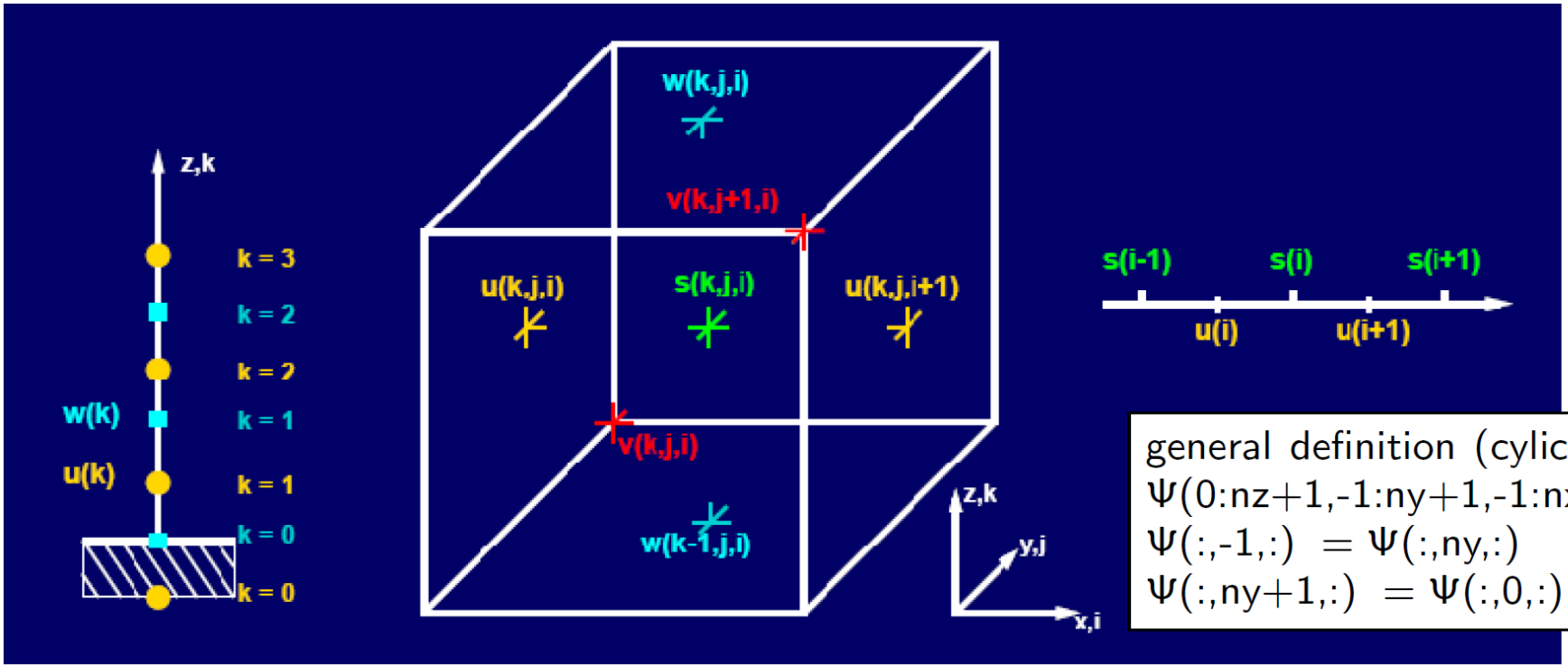
Overview

PALM is (almost) using simple, standard and fast numerical schemes:

- Spatial and temporal discretization by finite differences
- Explicit timestep schemes
  - Euler
  - Runge-Kutta, second- or third-order
- Advection schemes
  - Upstream
  - Piacsek-Williams (second order central finite differences)
  - Bott-Chlond-scheme (monotone, positiv definit, for scalars only)
  - 5th-order scheme of Wicker and Skamarock (as used in WRF model)
- Poisson-equation for dynamic pressure
  - Direct FFT-method
  - Multigrid-method
- Additional models embedded (Lagrangian particle model, radiation model,…)
- Boundary conditions:
  - Cyclic and non-cyclic horizontal boundary conditions
  - Special nesting boundary conditions
  - Constant flux layer with Monin-Obukhov similarity
  - Topography (full 3D)
  - Turbulent inflow (for non-cyclic horizontal boundary conditions)

Numerics & boundary conditions

Numerical grid



general definition (cylic):
$\Psi(0{:}nz+1,-1{:}ny+1,-1{:}nx+1)$
$\Psi(:,-1,:) = \Psi(:,ny,:)$
$\Psi(:,ny+1,:) = \Psi(:,0,:)$

- Equations are spatially discretized on an Arakawa-C grid.
- All scalar variables *s* (e.g. , *p*, e,* K_m, K_h) are defined at the cell centers.
- Velocity components (*u, v, w*) are shifted by half of the grid spacing.
- Spacings are equidistant, stretching along *z* is possible.
- Horizontal index bounds of variable arrays depend on the respective advection scheme

Timestep methods (I)

- **Euler:**

$$\frac{\partial \psi(t)}{\partial t} = F(\psi(t)) \rightarrow \frac{\psi(t+\Delta t) - \psi(t)}{\Delta t} \approx F(\psi(t)) \qquad u\frac{\Delta t}{\Delta x} = C < 1$$

for stability

$$\psi(t+\Delta t) = \psi(t) + \Delta t \cdot F(\psi(t)) \qquad \mathcal{O}(\Delta t)$$

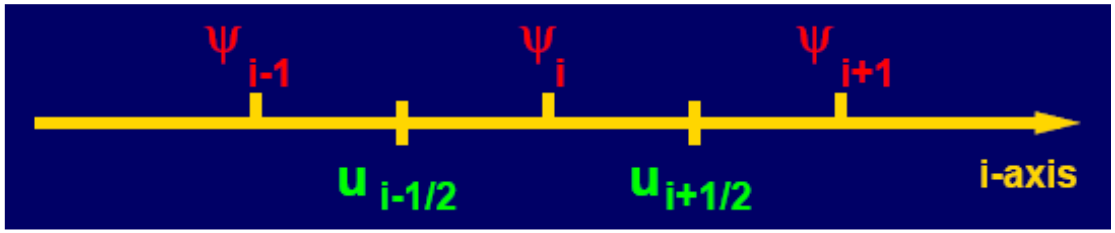
(used for SGS-TKE in special cases)

- **Runge-Kutta, third-order:**

$$k_1 = F(\psi(t))$$
$$k_2 = F\left(\psi(t) + \tfrac{1}{3}\Delta t \cdot k_1\right)$$
$$k_3 = F\left(\psi(t) - \tfrac{3}{16}\Delta t \cdot k_1 + \tfrac{15}{16}\Delta t \cdot k_2\right)$$
$$\psi(t+\Delta t) = \psi(t) + \tfrac{1}{30}\Delta t(5k_1 + 9k_2 + 16k_3) \qquad \mathcal{O}(\Delta t^2) \qquad C \leq 0.9$$

Advection methods (I)

- Piacsek Williams C3 (1970, J. Comput. Phy., 6, 392).
- Scheme of 2nd order accuracy.
- Conserves integrals of linear and quadratic quantities.
- Uses flux form of advection term, requires incompressibility.



$$\left.\frac{\partial(u\psi)}{\partial x}\right|_i = \frac{1}{2\Delta x}\left(u_{i+\frac{1}{2}}\psi_{i+1} - u_{i-\frac{1}{2}}\psi_{i-1}\right)$$

- In case of momentum advection (e.g. Ψ=u), $u_{i-1/2}$ and $u_{i+1/2}$ have to be obtained by linear interpolation.
- Shows strong dispersion in case of sharp gradients.
- May cause 2Δ wiggles and negative scalar concentrations in case of sharp gradients.

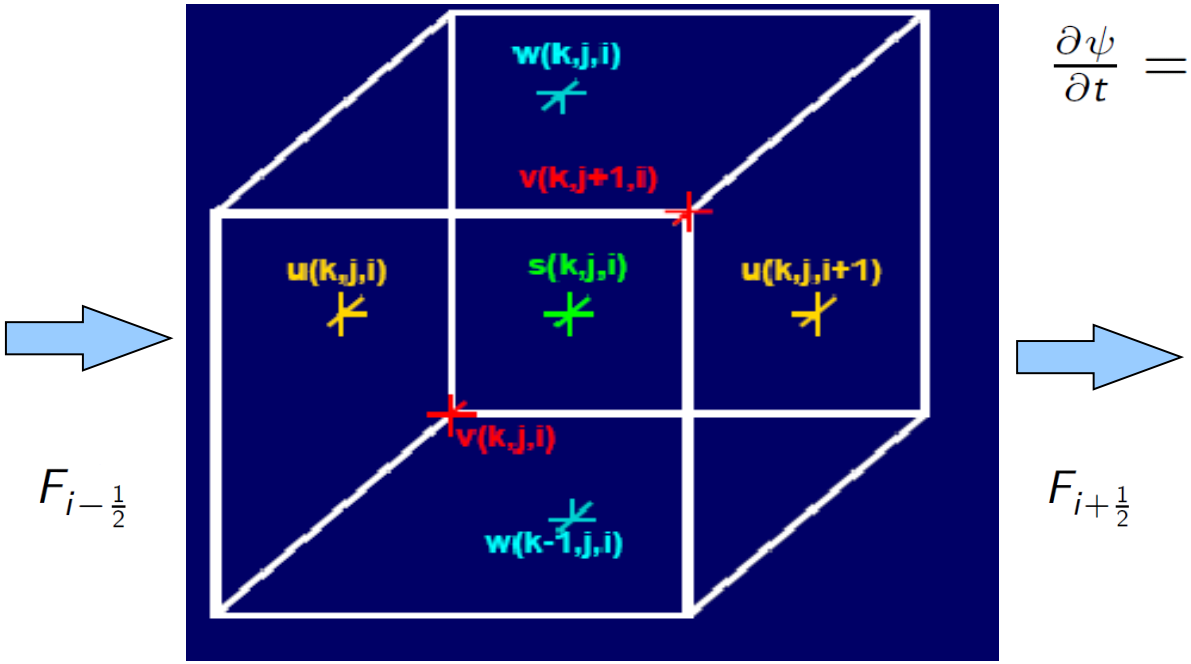
Numerics & boundary conditions

Advection methods (II)

- Bott-Chlond scheme
  - Chlond (1994).
  - Monotone, positive definit. Can only be used for scalars.
  - No negative scalar concentrations.
  - Conserves sharp gradients but makes them a bit smoother.
  - Numerically expensive.
  - Still not well optimized for use on cache-based machines.
  - Currently not applicable if topography is used.
- Default: Wicker and Skamarock scheme (5th order)
  - Much better accuracy than Piacsek Williams (less dispersive).
  - Much simpler algorithm than Bott-Chlond.
  - Requires additional ghost layers / memory (3).
  - Adds additional numerical dissipation.
  - Applicable with topography (order is degraded near walls).
  - Conserves integrals. Better spectral properties than Piascek-Williams.

Advection methods (III) – Wicker-Skamarock (I)

- Wicker and Skamarock (2002, Mon. Wea. Rev. 130, 2088 – 2097).
- 5th-order scheme, based on flux form of advection term.
- Difference of fluxes at the edge of the grid cell is used to discretize advection term.



$$\frac{\partial \psi}{\partial t} = -\nabla(u_i \psi) \approx -\frac{F_{i+\frac{1}{2}} - F_{i-\frac{1}{2}}}{\Delta x}$$

Advection methods (III) – Wicker-Skamarock (II)

Finite difference approximation of 6th order (strong Dispersion)

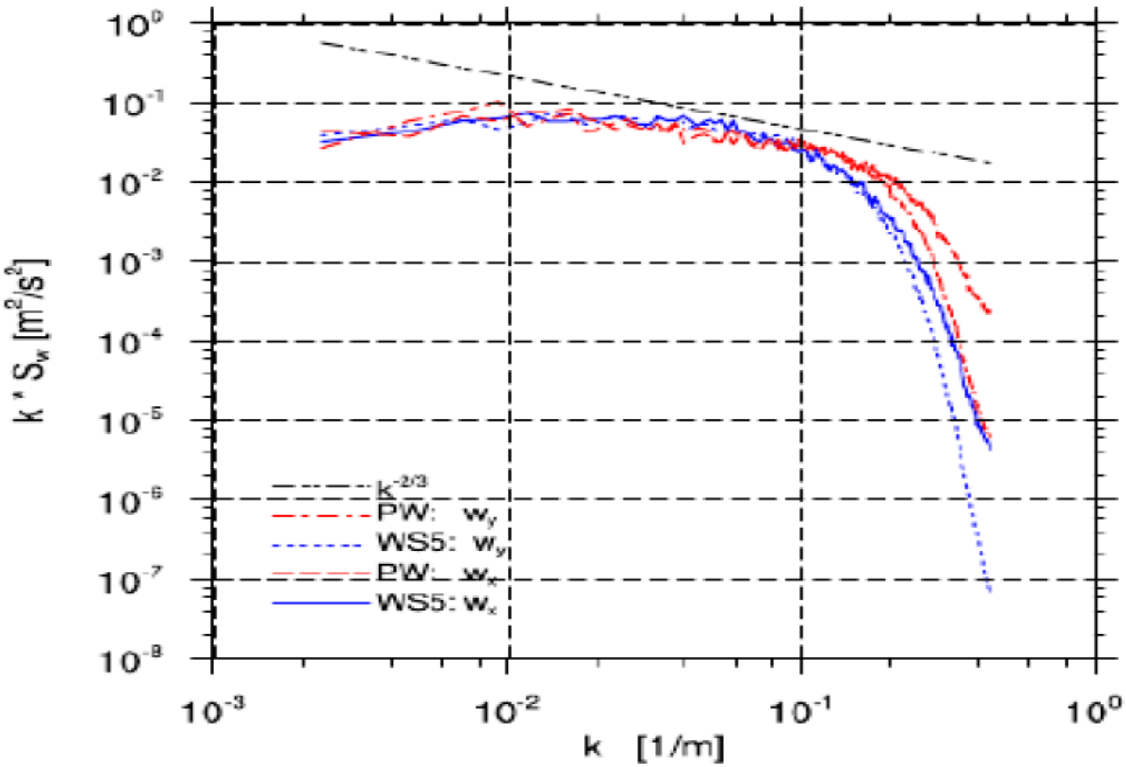
$$F^{6\text{th}}_{i-\frac{1}{2}} = \frac{1}{60} u_{i-\frac{1}{2}} \left(37(\Psi_i + \Psi_{i-1}) - 8(\Psi_{i+1} + \Psi_{i-2}) + (\Psi_{i+2} + \Psi_{i-3})\right)$$

Artificially added numerical dissipation term of 5th order

$$-\frac{1}{60} \left| u_{i-\frac{1}{2}} \right| \left(10(\Psi_i - \Psi_{i-1}) - 5(\Psi_{i+1} - \Psi_{i-2}) + (\Psi_{i+2} - \Psi_{i-3})\right)$$

Numerics & boundary conditions

Advection methods (III) – Wicker-Skamarock (V)



- Better resolution of larger scales (> 8 Δx) and hence less numerical energy transfer from larger to smaller scales compared to lower order schemes. Sharp gradients are well preserved for WS5.
- Less spectral energy at smaller scales.

Numerics & boundary conditions

Pressure solver (I)

- Governing equations of PALM require incompressibility.
- Incompressibility is obtained by a predictor-corrector method.

1. Momentum equations are solved without the pressure term giving a provisional velocity field which is not free of divergence.

$$\overline{u}_{i_{\mathrm{prov}}}^{t+\Delta t} = \overline{u}_i^t + \Delta t \left(-\frac{\partial}{\partial x_k} \overline{u}_k^t \overline{u}_i^t - \left(\varepsilon_{ijk} f_j \overline{u}_k^t - \varepsilon_{i3k} f_3 u_{\mathrm{g}_k} \right) + g \frac{\overline{\theta^*}^t}{\theta_0} \delta_{i3} - \frac{\partial}{\partial x_k} \overline{u_k' u_i'}^t \right)$$

predictor step

2. Assign all remaining divergences to the (perturbation) pressure p* so that the new corrected velocity field is the sum of the provisional, divergent field and the perturbation pressure term.

$$\overline{u}_i^{t+\Delta t} = \overline{u}_{i_{\mathrm{prov}}}^{t+\Delta t} + \Delta t \left(-\frac{1}{\rho_0} \frac{\partial \overline{p^*}^t}{\partial x_i} \right)$$

corrector step

3. The divergence operator is applied to this equation. Demanding a corrected velocity field free of divergence, this leads to a Poisson equation for the perturbation pressure.

$$\frac{\partial^2 \overline{p^*}^t}{\partial x_i^2} = \frac{\rho_0}{\Delta t} \frac{\partial \overline{u}_{i_{\mathrm{prov}}}^{t+\Delta t}}{\partial x_i}$$

4. After solving the Poisson equation, the final velocity field is calculated as given in step 2.

Numerics & boundary conditions

Pressure solver (II)

- **FFT-solver**
  - Discretization of the Poisson-equation by central differences.
  - 2D discrete FFT in both horizontal directions.
  - Solving the resulting tridiagonal set of linear equations.
  - Inverse 2D discrete FFT in both horizontal directions leading to the perturbation pressure.
  - Various FFT-methods are implemented.

- **Properties**
  - Very fast and accurate, O(n log n), n: number of gridpoints.
  - CPU requirement < 50% of total CPU time (except for large core numbers).
  - Due to non-locality of the FFT, the model parallelization requires transposition due to the domain decomposition (causes big communication overhead).
  - Requires periodic boundary conditions and uniform grids along x and y.

Pressure solver (III) – Multigrid method (I)

- **Iterative solver**
  - Based on Gauss-Seidel method (here for 2D-uniform grid).

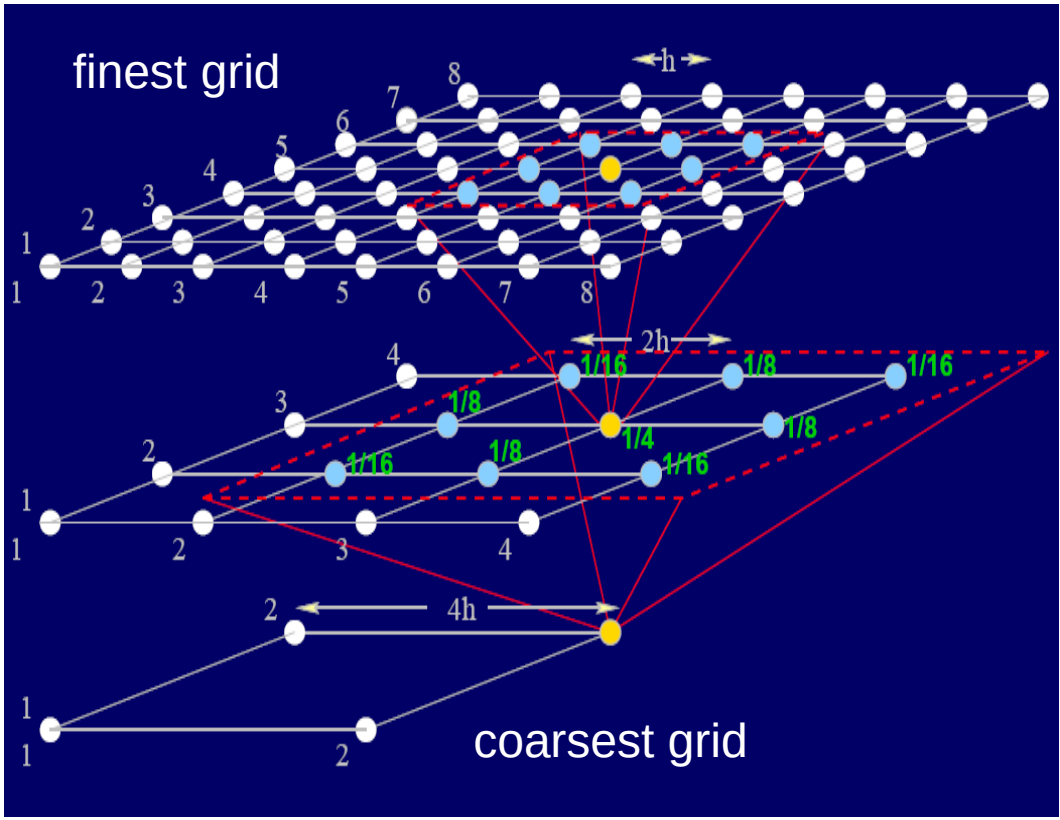
$$p_{i,j}^{k+1} = \frac{1}{4} \cdot \left(p_{i-1,j}^{k} + p_{i+1,j}^{k} + p_{i,j-1}^{k} + p_{i,j+1}^{k} - \Delta x^2 f(i,j,k) \right)$$

  - With each iteration step *k* the improved solution converges towards the exact solution.
  - Local scheme → information is required only from neighbouring points.
  - Very slow convergence.
  - Idea behind multigrid method: reduce divergence on grids with different grid spacing.
    - Local divergence is reduced on fine grid.
    - Larger scale divergence is reduced on coarser grids.

Numerics & boundary conditions

Pressure solver (III) – Multigrid method (II)

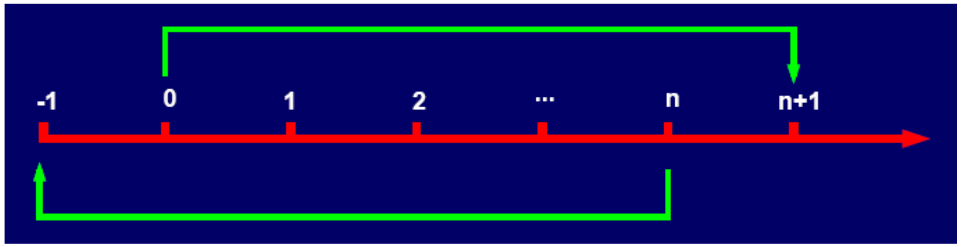
- On each grid-level an approximate solution of the Poisson equation is obtained performing a few iterations.
- The solution is transmitted to the next coarser grid-level where it is used as the first guess to solve the equation.
- This procedure is performed up to the coarsest grid-level, ideally containing two grid-points in each direction.
- From the coarsest grid-level the procedure is passed in backward order to get the final solution.
- For large grids faster than FFT method. V- and W-cycles are implemented.



Boundary conditions (I)

Lateral (xy) boundary conditions:

- Cyclic by default, allowing undisturbed evolution / advection of turbulence.



$$\begin{aligned} \Psi(-1) &= \Psi(n) \\ \Psi(n+1) &= \Psi(0) \end{aligned}$$

- Dirichlet (inflow) and radiation (outflow) conditions are allowed along either x- or y-direction.
- Mesoscale nesting and self-nesting allow Dirichlet conditions simultaneously at all boundaries.
- In case of a Dirichlet condition, the inflow is laminar (by default) and the domain has to be extended to allow for the development of a turbulent state, in general, or the synthetic turbulence generator has to be applied.

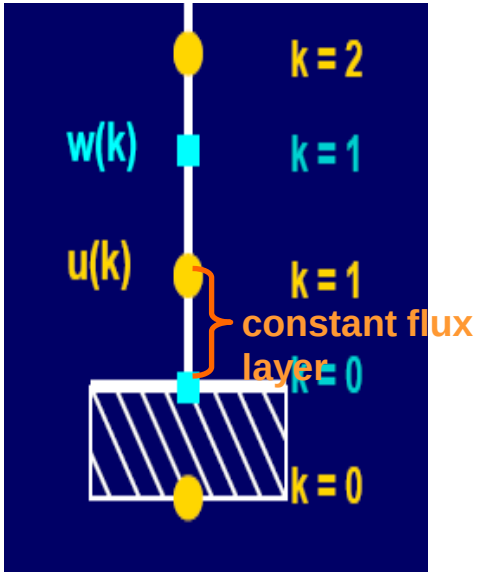
Boundary conditions (II)

Surface boundary condition (I):

- Monin-Obukhov-similarity is used by default, i.e. a constant flux layer is assumed between the surface and the first grid layer. **Turbulence is completely parameterized here!**

$$\frac{\partial \overline{u}}{\partial z} = \frac{u_*}{\kappa z}\Phi_{\mathrm{m}}; \qquad u_* = \sqrt{-\overline{w'u'_0}} = \sqrt{\frac{\tau_0}{\overline{\rho}}}$$

$$\frac{\partial \overline{\theta}}{\partial z} = \frac{\vartheta_*}{\kappa z}\Phi_{\mathrm{h}}; \qquad \vartheta_* = \frac{\overline{w'\theta'_0}}{u_*}$$



- Integration between $z=z_0$ (roughness height) and $z=z_p$ (top of constant flux layer, k=1) gives the only unknowns u_* and θ_* which then define the surface momentum and heat flux, used as the real boundary conditions.

- Φ_m, Φ_h: Dyer-Businger functions

$$\Phi_{\mathrm{m}} = \begin{cases} 1 + 5\,\mathrm{Rif} & \text{stable} \\ 1 & \text{neutral} \\ (1 - 16\,\mathrm{Rif})^{-1/4} & \text{unstable} \end{cases}$$

$$\mathrm{Rif} = \frac{\frac{g}{\overline{\theta}}\overline{w'\theta'_0}}{\overline{w'u'}\frac{\partial \overline{u}}{\partial z}}$$

Richardson flux number

Boundary conditions (III)

Surface boundary condition (II):

- Monin-Obukhov-similarity is only valid for a horizontally homogeneous infinite surface. **(!)**
- The surface temperature has to be given as boundary condition.
- Alternatively, the surface heat flux can be prescribed, or a surface energy balance equation (including a soil model) can be solved to get the surface temperature (included in the land- and urban-surface model).
- Instead of MO-similarity, no-slip conditions or free-slip conditions can be used.
- Pressure boundary condition: $\frac{\partial p}{\partial z} = 0$ in order to guarantee

  $w(z = 0) = 0$
- SGS-TKE condition: $\frac{\partial e}{\partial z} = 0$

Boundary conditions (IV)

Boundary conditions at the top (default)

- Dirichlet conditions for velocities: $u = u_{\mathrm{g}}, \quad v = v_{\mathrm{g}}, \quad w = 0$
- Neumann conditions (temporal constant gradients) for scalars:

$$\frac{\partial \theta}{\partial z} = \left.\frac{\partial \theta}{\partial z}\right|_{t=0}$$

- Pressure: Neumann $\frac{\partial p}{\partial z} = 0$
- SGS-TKE: Neumann $\frac{\partial e}{\partial z} = 0$
- A damping layer can be switched on in order to absorb gravity waves.

Numerics & boundary conditions

Initial conditions

- All 3D-arrays are initialized horizontally homogeneous with vertical profiles.
- Three different profile types can be chosen:
  - constant (piecewise linear) profiles

$$\textbf{e.g.}\ \ u = 0,\, v = 0,\, \frac{\partial \theta}{\partial z} = 0\ \textbf{up to}\ z = 1000\,\mathrm{m},\ \frac{\partial \theta}{\partial z} = +1.0\ \textbf{up to top}$$

  - velocity profiles calculated by a 1D-model (which is a part of PALM)

    a stationary (piecewise linear) temperature profile is assumed for the 1D-model

  - arbitrary wind profiles (e.g. from observations)
- In case of mesoscale nesting wind profiles can be provided via the dynamic driver file

Under horizontally homogeneous initial conditions, random fluctuations have to be added in order to initiate turbulence generation!